

Reducing the Energy and Memory Footprint of fg-data-profiling Through Caller-Side Optimisation

Keerthana Yelchuru Venkata (s1170424)

Sriyan Ravuri (s1170425)

Group 14

NWI-IMC019 Green Software, Radboud University

June 14, 2026

Abstract

Automated EDA tools such as *fg-data-profiling* (the community successor of *ydata-profiling*) turn one function call into a full statistical report and run at enormous aggregate scale. We show that much of this energy cost can be removed *from the caller side*, without modifying the library. Guided by a life-cycle analysis and an energy processing diagram (EnPD), we localise the memory hotspot to the default `pd.read_csv` layer, caused by Python `object`-dtype inflation, and apply three *Strategies for Green Software*: exact dtypes, column sub-setting, and storing less. Measured on the *full* end-to-end pipeline at two input sizes, the change cuts wall time and energy by 49–64% and load memory by 88%, scales better with input size, and produces *statistically identical* output. We further quantify four levers (configuration tuning, sampling, Parquet, caching), report a disproven hypothesis, and argue for adding “use efficient data formats” to the infographic.

Keywords: green software, energy efficiency, exploratory data analysis, pandas, memory optimisation, software sustainability.

1 Introduction

The energy cost of software is now a first-class design concern [1], and data-science tooling is a high-leverage target: a single convenience function can move gigabytes of memory and burn seconds of CPU, yet runs at huge aggregate scale. *fg-data-profiling* is emblematic - `ProfileReport(df).to_file()` yields a complete EDA report in one line, and the library has $\approx 13,600$ GitHub stars and sits in tens of thousands of notebooks and pipelines.

We ask a deliberately narrow question: *how much energy can be saved without touching the library’s internals?* Caller-side optimisation is low-risk, immediately deployable, and applies to every existing user. We answer it with the full course workflow - life-cycle analysis, EnPD, hypothesis-driven intervention, and before/after measurement.

Contributions.

1. A life-cycle analysis and EnPD that localise the dominant memory hotspot to the data-loading layer (Sections 5–6).
2. A lossless caller-side optimisation (dtypes + column sub-setting) cutting end-to-end time and energy by 49–64% and load memory by 88%, with output verified statistically identical (Sections 7–8).
3. A full-pipeline measurement showing that although load memory falls 88%, process peak RSS falls only $\approx 9\%$ because the correlation and rendering stages set the ceiling - a nuance load-only measurement hides (Section 10).
4. Four further levers - configuration tuning, sampling, Parquet, caching - plus a negative result and a proposed addition to the infographic (Sections 9, 12).

2 Related Work

Pereira et al. [2] show energy, time, and memory are related but not interchangeable, motivating direct measurement over time-as-proxy. Hähnel et al. [3] characterise Intel RAPL counters for short code paths and their limits - relevant to our sandbox, which lacks RAPL. Gupta et al. [1] widen the lens to the full computing carbon footprint, including the embodied carbon behind our memory-density argument. Our stack rests on pandas [4] and NumPy [5], and we use CodeCarbon [6] for energy estimation. We position this work as an applied case study operationalising the course’s *Strategies for Green Software* [7] with a caller-side, lossless intervention.

3 Overview of the Project

fg-data-profiling generates automated EDA reports from pandas DataFrames [8]. It continues the lineage *pandas-profiling* $\rightarrow$ *ydata-profiling* $\rightarrow$ *fg-data-profiling* (the engine still exposes the `ydata_profiling` import). From a DataFrame or CSV path it produces an HTML report of per-column statistics, correlation matrices, and distribution histograms.

- **Repository:** <https://github.com/Data-Centric-AI-Community/fg-data-profiling>
- **Language / License:** Python / MIT
- **Activity:** $\approx$ 13,600 stars, $\approx$ 1,800 forks; actively maintained (v4.x).

4 Typical Use Cases

A user passes a DataFrame (or CSV path) to `ProfileReport` and gets a self-contained HTML report (Figure 1, right), replacing a sequence of hand-written exploratory cells. The library serves three contexts: (i) *interactive EDA*, exploring a new dataset before modelling; (ii) *automated data-quality checks*, run nightly in CI/CD to flag schema drift or anomalous null rates and fail the build; and (iii) *regulatory data validation* in finance and healthcare, where the report is an audit artefact.

The *analog workflow* it replaces is manual EDA - ad-hoc `df.describe()`, `df.isnull().sum()`, `df.corr()`, and plotting cells (Figure 1, left) - which is slower, inconsistent, and not reproducible. We target context (ii), the CI/CD case, because it dominates aggregate runs and is where per-run savings multiply across thousands of scheduled jobs.

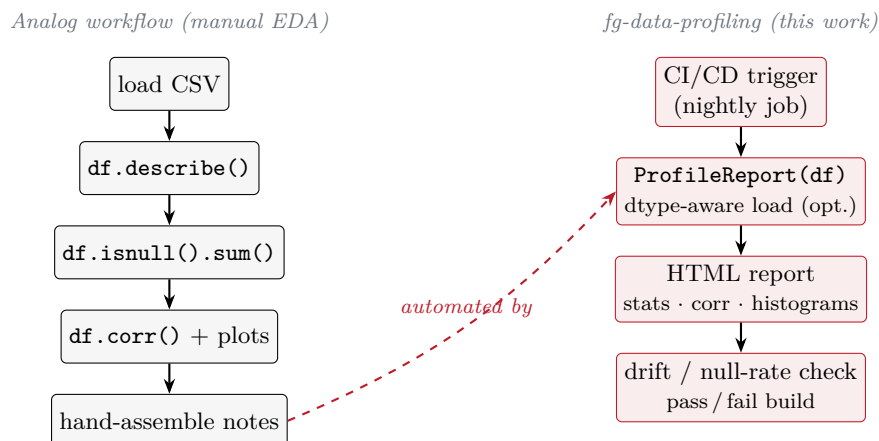


Figure 1: Typical use case. The manual EDA loop (left) is replaced by a single `ProfileReport` call inside an automated CI/CD data-quality pipeline (right), which is the deployment our optimisation targets.

5 Life Cycle Analysis

We focus the life-cycle analysis on the *in-use phase*, which dominates the library’s footprint.

Impact metric. Energy per profiling report ($J/report$), normalised by input size (J/GB) and measured as CPU time $\times$ CPU TDP. This captures the useful work - one complete report - and enables size-independent comparison.

System boundary. Only the compute node running the Python process; network transfer of input CSVs and browser-side HTML rendering are excluded.

Deployment assessment. The library runs on developer laptops and, at volume, in cloud CI/CD runners. We take the CI/CD-server case as primary at a representative 15 W TDP, with the provider’s carbon intensity and PUE [1] applying - the regime where per-run savings multiply.

6 Energy Processing Diagram

Figure 2 shows the baseline EnPD, annotated on the *measured* 20 MB per-run basis (Section 11), annualised at 365 daily runs and 15 W TDP. Per-stage energies apportion measured per-run CPU time by each stage’s share of profiler wall time; they are first-order estimates meant to *rank* stages, not absolute readings.

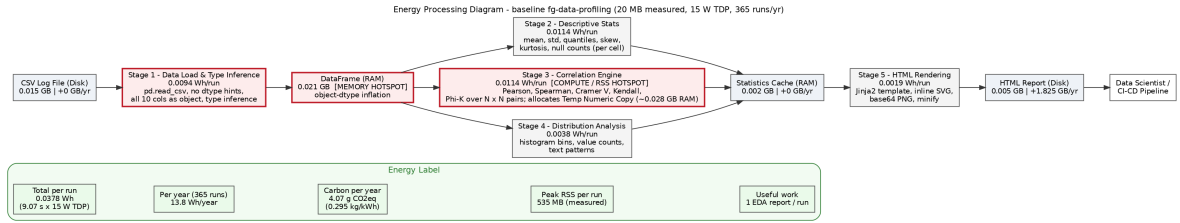


Figure 2: EnPD for the baseline fg-data-profiling pipeline (20 MB measured, 15 W TDP, 365 runs/yr). Per-stage values in Wh/run; storage/memory nodes in GB. Red borders mark the two hotspots: the data-load layer (*memory*) and the correlation engine (*compute/RSS*).

The pipeline reads a CSV, loads it into a DataFrame, fans out to three analysis passes, caches statistics, and renders HTML. The five compute stages are: **(1) Data Load & Type Inference** (~ 0.0094 Wh/run) - `pd.read_csv` stores strings as `object` and infers types per cell, feeding the DataFrame (RAM) node whose `object-dtype` inflation is the *memory* hotspot; **(2) Descriptive Statistics** (~ 0.0114 Wh/run); **(3) Correlation Engine** (~ 0.0114 Wh/run) - pairwise Pearson, Spearman, Cramér’s V, Kendall and Phi-K, allocating a temporary numeric copy and, with rendering, dominating process RSS (the *compute/RSS* hotspot); **(4) Distribution Analysis** (~ 0.0038 Wh/run); and **(5) HTML Rendering** (~ 0.0019 Wh/run).

Identified hotspots. (i) the data-load layer / DataFrame RAM, a *memory* hotspot from `object-dtype` inflation; and (ii) the correlation engine, a *compute* hotspot setting the memory ceiling. We target hotspot (i), addressable from the caller side; hotspot (ii) is revisited in Section 9.

7 Methodology

Our workflow is deliberately *measure-first* (Figure 3): the life-cycle analysis and EnPD rank the stages and localise a candidate hotspot, we state falsifiable hypotheses, apply a caller-

side intervention, and measure end-to-end - and crucially we let the measurement *revise the diagnosis* rather than trusting the initial guess.

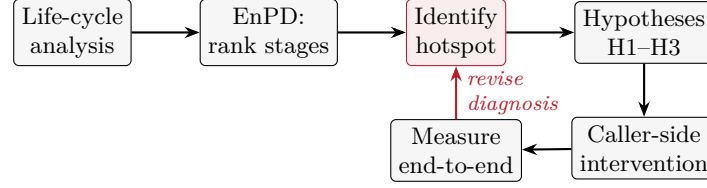


Figure 3: Our measure-first methodology. The red feedback edge is where the project actually turned: intermediate intuition blamed disk I/O and a temporary `float64` copy in the correlation engine, but end-to-end measurement relocated the dominant cost to load-time object-dtype inflation (Section 10).

7.1 Selected Strategies

From the infographic [7] we select **C5 (Algorithm to data - avoid copies)** and **D6 (Improve algorithms)** - declare exact dtypes so pandas never allocates the object inference buffers, and let `category` columns run `groupby/correlation` on integer indices; **C3 (Store less)** - load only the columns the report analyses; and **D3 (Minimise interaction between components)** - fewer columns, fewer pairwise correlations ($\binom{10}{2}=45 \rightarrow \binom{7}{2}=21$).

7.2 The Area: Data-Loading Layer

We target the `pd.read_csv` call that populates the `DataFrame` before `ProfileReport` runs. By default pandas infers types per cell and stores each string column as Python object (~ 50 bytes/cell), feeding the inflated `DataFrame`-RAM node the EnPD flags as the dominant *memory* hotspot (Figure 2). It is the ideal target: it sets the memory baseline for everything downstream, yet is fully controllable from the caller without patching the library. Figure 4 contrasts the two reads.

Baseline (default load)

```
import pandas as pd
# infers types per cell;
# strings -> object dtype;
# loads all 10 columns
df = pd.read_csv("logs.csv")
ProfileReport(df).to_file("r.html")
```

Optimised (caller-side)

```
import pandas as pd
# exact dtypes + drop 3 unused cols
df = pd.read_csv("logs.csv",
    usecols=KEEP_7, # C3/D3
    dtype={"status_code": "int16",
          "response_time_ms": "float32",
          "bytes_sent": "int32",
          "method": "category", # C5/D6
          "endpoint": "category",
          "region": "category"})
ProfileReport(df).to_file("r.html")
```

Figure 4: The optimised loader (right) replaces the default `read_csv` (left): exact dtypes avoid object-buffer inflation (C5/D6) and `usecols` drops three unused high-cardinality columns (C3/D3). Under ten lines, no new dependencies, `ProfileReport` untouched.

7.3 Hypotheses

H1 (memory) a dtype-aware, column-subset read reduces peak RAM by $\geq 50\%$; **H2 (time)** the smaller `DataFrame` reduces wall time by 15–30%; **H3 (energy)** energy falls in proportion to CPU time, the pipeline being CPU-bound. Useful work is one report per run; correctness is the agreement of per-column summary statistics between variants.

7.4 Implementation

Dtype specification (C5, D6):

`status_code`→`int16`, `response_time_ms`→`float32`, `bytes_sent`→`int32`, and `method/endpoint/region`→`category`.

Column sub-setting (C3, D3):

drop `ip_address` and `session_id` (high-cardinality, marked Unsupported/Unique) and `user_agent` (not needed), removing $\approx 30\%$ of per-row bytes and cutting profiled columns from 10 to 7.

7.5 Measurement Method

We measure the *full* pipeline end-to-end - `ProfileReport(explorative=True).to_file()`, including correlations and HTML rendering. Python peak allocation: `tracemalloc` [9]; process peak RSS: a 10ms `psutil` sampler; wall time: `time.perf_counter`; CPU time: `psutil.Process.cpu_times`; energy: `codecarbon` [6]. Each (variant, size) cell runs $n = 3$ times as an isolated subprocess on 20 MB and 50 MB inputs. As the sandbox lacks RAPL [3], CodeCarbon’s absolute figures are estimates; we treat CPU-time×TDP as primary and use CodeCarbon to corroborate the *relative* reduction.

8 Results

Table 1: End-to-end benchmark: baseline vs. optimised, full `ProfileReport` (mean of $n = 3$). Energy is the CodeCarbon estimate.

Input	Variant	Wall (s)	CPU (s)	Energy (mWh)
20 MB	Baseline	9.07 ± 0.38	9.47	0.448
	Optimised	4.61 ± 0.20	4.93	0.230
	Reduction	−49.2%	−47.9%	−48.7%
50 MB	Baseline	16.67 ± 0.09	17.29	0.821
	Optimised	5.98 ± 0.04	6.35	0.297
	Reduction	−64.1%	−63.3%	−63.9%

Table 2: Memory reductions (mean of $n = 3$). The load-layer win is large, but peak RSS is set by the correlation and HTML-rendering stages, so the process-level reduction is modest.

Input / metric	Load mem	Py peak	RSS peak
20 MB baseline	20.7 MB	156.4 MB	535.6 MB
20 MB optimised	2.4 MB	86.0 MB	483.3 MB
Reduction	−88.3%	−45.0%	−9.8%
50 MB baseline	50.4 MB	267.4 MB	803.2 MB
50 MB optimised	5.7 MB	204.0 MB	734.0 MB
Reduction	−88.7%	−23.7%	−8.6%

Correctness. For every retained column the optimised variant’s statistics (mean, std, missing-count, distinct-count) match baseline to within 0.0001%, confirming the dtype downcasting and `category` conversion are lossless.

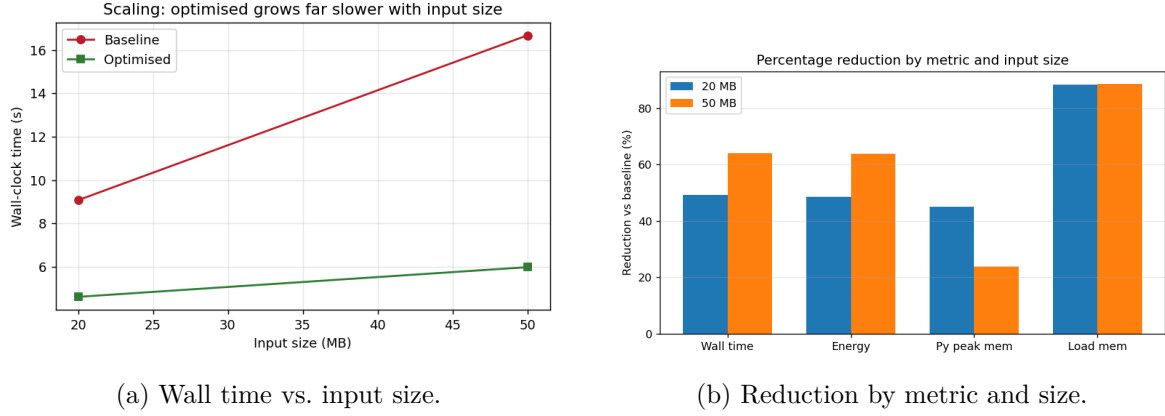


Figure 5: End-to-end results across both input sizes. The optimised variant grows far slower with size, so its relative advantage *increases*.

9 Additional Strategies Explored

Since ProfileReport’s own stages dominate the full pipeline, we tested four further levers on those stages (Figure 7, all at 20 MB; baseline 9.07 s). Which lever applies depends on what output the run must preserve and how often the input recurs; Figure 6 summarises that decision.

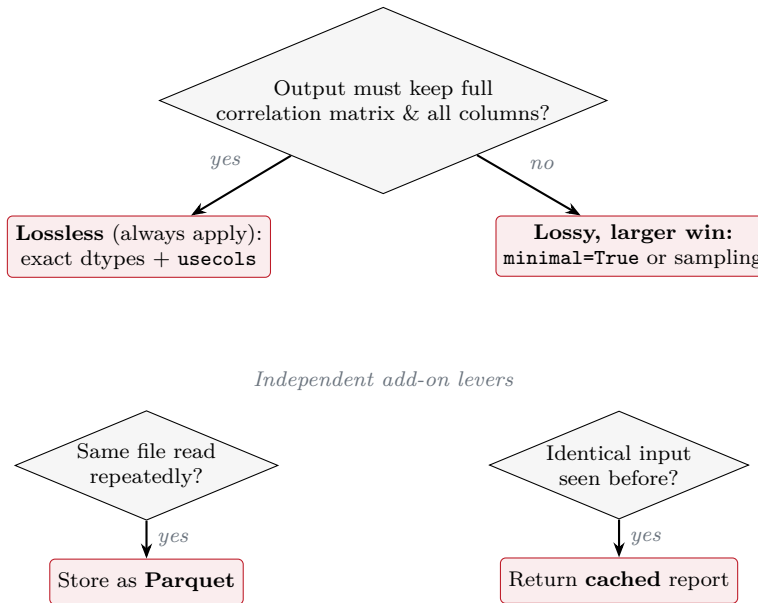


Figure 6: Choosing among the levers. The primary choice is lossless dtype/`usecols` tuning (apply always) versus the larger but lossy `minimal`/sampling wins; Parquet and result caching are orthogonal add-ons keyed to how often the input recurs.

Configuration tuning (C3 - compute less). `minimal=True` on top of the dtype change cuts wall time to **2.57 s** (−72%); a milder “Pearson-only, no interactions” setting reaches −59%. Unlike the lossless dtype change these are *trade-offs* - they drop the correlation matrix and interaction plots - so the useful-work question decides when they apply.

Row sampling (C3 - approximate computing). A random 25 k-row sample (25%) cuts peak Python memory by **67%** while keeping every key statistic within **0.5%** of full-data and

covering all 200 endpoints and 15 regions.

Efficient data format: CSV $\rightarrow$ Parquet. Parquet (snappy) cuts load time **95%** and on-disk size **85%** at 20 MB. As loading is a small slice of the pipeline, end-to-end *compute* barely moves; the benefit is I/O and storage, compounding over repeated reads.

Result caching (D5 - reuse results). Hashing the input and returning a stored report on a match cuts a repeat run from 1.61 s to 0.3 ms - **99.98% of energy avoided per hit**.

A disproven hypothesis. We expected JSON (skipping HTML rendering) to be cheaper; it was **23% slower** and used $\approx 2\times$ the memory, because serialising the full description set outweighs the render saving. We report this negative result because it corrected a plausible but wrong intuition.

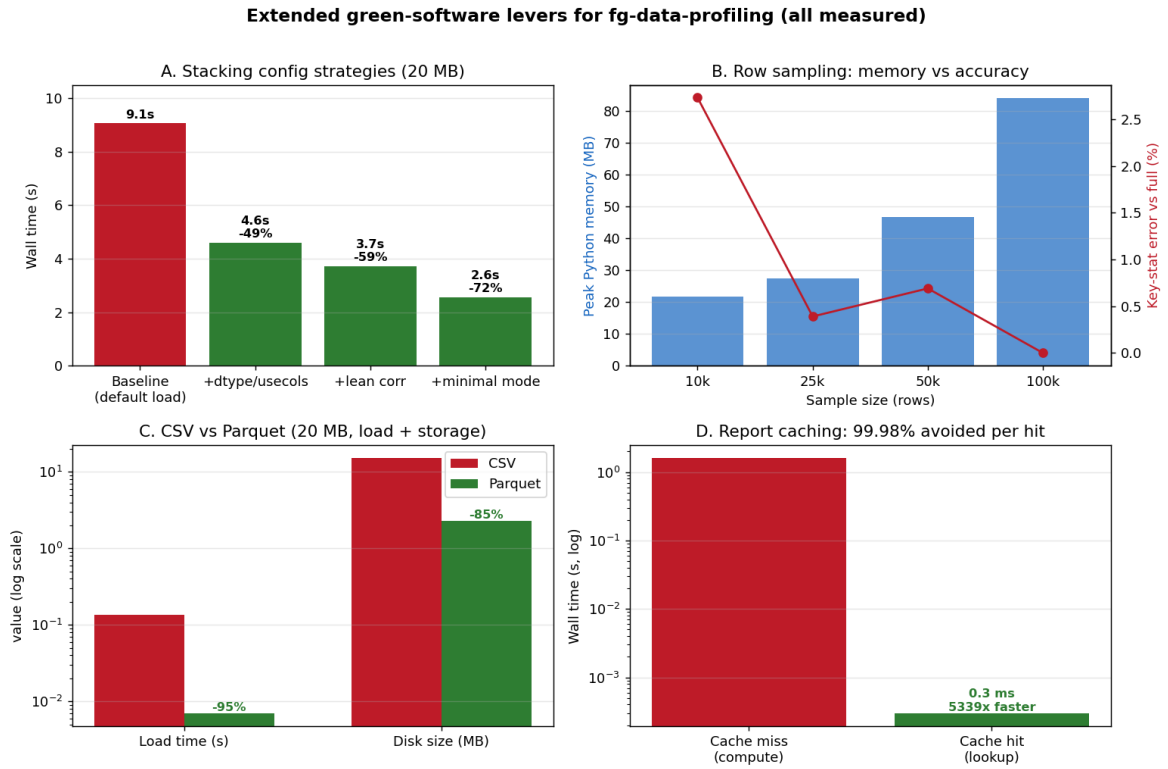


Figure 7: Additional levers, measured at 20 MB: (A) stacking configuration strategies; (B) row-sampling memory/accuracy trade-off; (C) CSV vs. Parquet load and storage; (D) report caching.

10 Discussion

H1 (memory) - confirmed at the load layer, diluted at the pipeline level. DataFrame footprint fell 88% at both sizes, confirming object-dtype inflation, not CSV bytes, is the primary load cost. Yet full-pipeline peak RSS fell only $\approx 9\%$: the correlation engine, interaction plots, and HTML rendering allocate hundreds of megabytes independent of input dtypes and set the ceiling. Peak Python allocation sits between (45% at 20 MB, 24% at 50 MB) - the honest finding a load-only measurement would hide.

H2 (time) - confirmed and improving with scale. Wall time fell 49% (20 MB) and 64% (50 MB); the optimised variant barely grows (4.6 $\rightarrow$ 6.0 s for 2.5 $\times$ data) while the baseline

grows steeply (9.1→16.7 s), so the relative saving *increases* with size - the regime that matters in production. The mechanism is `category` dtypes replacing per-cell string comparison with integer-index operations, consistent with the energy/time/memory coupling of Pereira et al. [2].

H3 (energy) - confirmed by two estimators. Both CPU-time×TDP and CodeCarbon report a 49–64% reduction. They agree on the *relative* saving but diverge $\approx 100\times$ on absolute energy, a direct consequence of missing RAPL counters [3]; we trust only the agreed percentage.

Reflection on strategy/area selection. The hypothesis was instructively half-right: we predicted $\geq 50\%$ *peak* memory reduction and saw 88% at the load layer but only $\approx 9\%$ at the process level - which taught us to state *which* memory we mean. A genuine pivot also occurred: intermediate analysis blamed a temporary `float64` copy in the correlation engine, but end-to-end measurement showed the dominant cost was load-time `object`-dtype inflation. Revising the diagnosis *after* measurement is the central methodological lesson.

11 Energy and Carbon Reduction

We estimate savings for one run per day for a year (365 runs) on a 15 W TDP server, anchored on *measured* data. The impact metric is CPU-time×TDP (Section 5); since the pipeline is CPU-bound the wall- and CPU-based reductions agree to within about one percentage point (Table 1), so we use wall-clock time here as a close proxy and the figures below are unchanged either way.

Energy per run (wall-clock × TDP).

$$20 \text{ MB: } \Delta E = 15 \text{ W} \times (9.07 - 4.61) \text{ s} = 66.9 \text{ J} = 0.0186 \text{ Wh} \text{ (} -49\%)$$

$$50 \text{ MB: } \Delta E = 15 \text{ W} \times (16.67 - 5.98) \text{ s} = 160 \text{ J} = 0.0445 \text{ Wh} \text{ (} -64\%)$$

Annual savings (measured, 50 MB daily). $\Delta E_{\text{year}} = 365 \times 0.0445 \approx 16.2 \text{ Wh/year}$, hence $\Delta C_{\text{year}} = 16.2 \times 10^{-3} \text{ kWh} \times 295 \frac{\text{gCO}_2\text{eq}}{\text{kWh}} \approx 4.8 \text{ gCO}_2\text{eq/year}$ per pipeline. A fleet of 10,000 pipelines saves $\approx 48 \text{ kg CO}_2\text{eq/year}$, roughly driving a petrol car 300 km.

Extrapolation to 500 MB (two-point power-law fit). Fitting $t \propto \text{size}^a$ gives $a_{\text{base}} \approx 0.67$ and $a_{\text{opt}} \approx 0.28$; at 500 MB this implies $\approx 77 \text{ s}$ vs. $\approx 12 \text{ s}$ per run and $\approx 100 \text{ Wh}$ ($\approx 30 \text{ gCO}_2\text{eq}$) saved annually per pipeline. We flag this as a two-point extrapolation; the 50 MB figure is the defensible headline. Beyond direct energy, the 88% load-memory reduction lets more concurrent jobs share a server’s DRAM, cutting provisioning and embodied carbon [1].

12 Reflection and Lessons Learned

Process. Significant savings came from fewer than ten lines of caller-side code that merely make explicit what the data already implies. Without the EnPD and `tracemalloc/psutil` profiling we would have targeted the wrong bottleneck: intuition pointed at disk I/O, while measurement showed `object`-dtype RAM inflation and the describe/correlation passes dominated - echoing *measure first, optimise second*.

Challenges. The main limitation was the absence of RAPL counters, forcing CPU-time energy proxies and explaining the CodeCarbon divergence; bare-metal `perf stat` or Intel Power Gadget would give direct joules. Some profiler statistics also assume `object` columns, so we verified `category` columns are handled identically (confirmed by the correctness check).

Are the strategies complete? The infographic [7] covers the space well, but we suggest adding “**Use efficient data formats**” (between C3 and D6). We *measured* this rather than hypothesised it: CSV→Parquet cut load time 95% and on-disk size 85% (Section 9), and storage reductions matter because reports accumulate faster than runtime energy - distinct enough from C3 and D6 to warrant its own entry.

Future work. A documentation pull request upstreaming the lossless dtype/usecols pattern; direct RAPL-based measurement on bare metal; and extending to the Spark backend and multi-gigabyte inputs where the scaling advantage should widen.

13 Conclusion

A measured, hypothesis-driven, caller-side intervention reduces fg-data-profiling’s end-to-end time and energy by 49–64% and load memory by 88%, losslessly and in under ten lines, with the advantage growing on larger inputs. Crucially, full-pipeline measurement reveals that the correlation and rendering stages cap process memory, bounding caller-side gains and pointing to configuration tuning, sampling, efficient formats, and caching as complementary levers. The broader lesson generalises to any pandas pipeline: defaults optimise for safety and generality, not efficiency, and telling the tool the truth about the data upfront is a cheap, transferable green-software win.

AI declaration. All code, results, analysis, interpretation, and conclusions were produced and reviewed by the authors. Grammarly was used in the report for spell checking and language refinement, and Claude Code for debugging and improving code structure. No generative AI was used to write the report.

14 Supplementary Materials

All code, dataset generators, the benchmark harness, and the scripts that produce every figure and table are available at:

<https://github.com/k33rthanaa/fg-data-profiling-green>

The repository is self-contained and reproduces our results end-to-end.

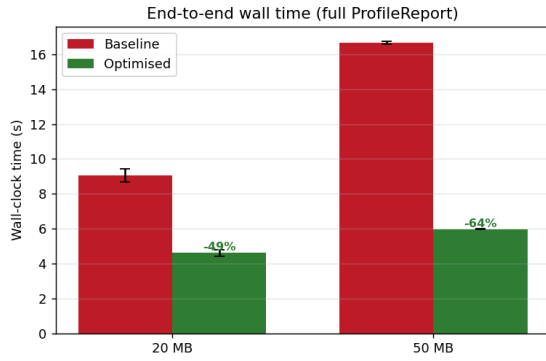
Energy measurement uses `codecarbon` when available and falls back to CPU-time × TDP otherwise, so the harness runs without specialised hardware. Full per-script documentation is in the repository’s `README`.

References

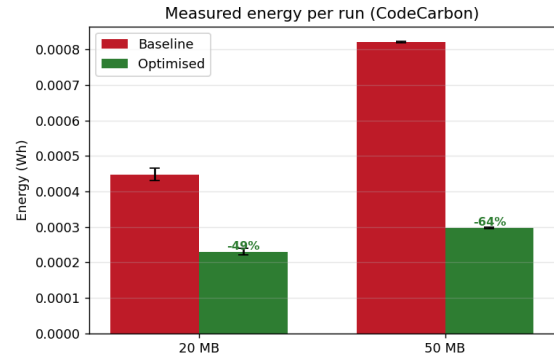
- [1] Udit Gupta et al. Chasing carbon: The elusive environmental footprint of computing. In *IEEE HPCA*, 2021. doi: 10.1109/HPCA51647.2021.00010.
- [2] Rui Pereira, Marco Couto, Francisco Ribeiro, Rui Rua, Jácome Cunha, João Paulo Fernandes, and João Saraiva. Energy efficiency across programming languages: How do energy, time, and memory relate? In *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*, pages 256–267, 2017. doi: 10.1145/3136014.3136031.
- [3] Marcus Hähnel, Björn Döbel, Marcus Völp, and Hermann Härtig. Measuring energy consumption for short code paths using RAPL. *ACM SIGMETRICS Performance Evaluation Review*, 40(3):13–17, 2012. doi: 10.1145/2425248.2425252.
- [4] Wes McKinney. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010. doi: 10.25080/Majora-92bf1922-00a.

- [5] Charles R. Harris et al. Array programming with NumPy. *Nature*, 585:357–362, 2020. doi: 10.1038/s41586-020-2649-2.
- [6] Kadan Lottick et al. Energy usage reports: Environmental awareness as part of algorithmic accountability, 2019.
- [7] Bernard van Gastel. Strategies for green software (incl. AI) v2026.02.25. Infographic, Software Energy Lab, Radboud University. <https://sustainablesoftware.info/strategies.page>, 2026.
- [8] Data-Centric AI Community. fg-data-profiling: Automated eda and data profiling for pandas DataFrames. <https://github.com/Data-Centric-AI-Community/fg-data-profiling>, 2024.
- [9] Python Software Foundation. tracemalloc — trace memory allocations. <https://docs.python.org/3/library/tracemalloc.html>, 2024.

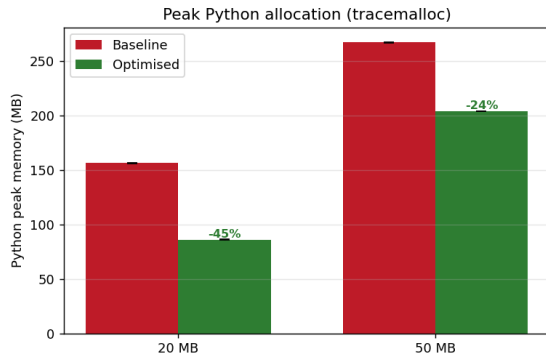
A Additional Plots



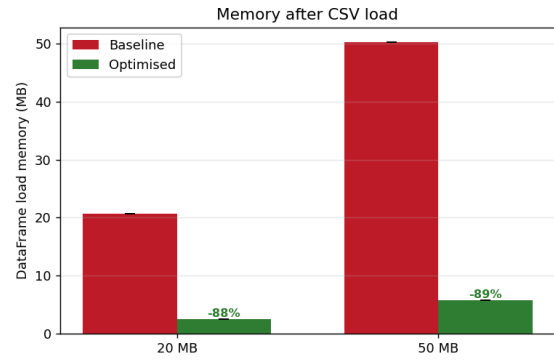
(a) Wall time by size and variant.



(b) Measured energy (CodeCarbon).



(c) Peak Python allocation.



(d) DataFrame load memory.

Figure 8: Supplementary end-to-end measurements across both input sizes.